Nesneye Yönelik Programlama
Hafta 4

Eliminate Duplicates (Tekrarları Eleme)

Verilen bir dizideki tekrar eden elemanları silen bir metot yazmamız beklenmektedir.
Yani dizideki "2" sayısından 3 tane varsa, yeni oluşturulacak dizide sadece bir tane "2" yer almalıdır.
Metot, orijinal diziyi değiştirmeyecek, içinde sadece benzersiz elemanların bulunduğu yeni bir dizi döndürecektir.

Çözüm Mantığı:

1. Geçici bir dizi oluşturup, orijinal dizideki elemanları sırayla kontrol ederiz.
2. Elemanı geçici diziye eklemeden önce, "Bu eleman daha önce geçici diziye eklendi mi?" diye bakarız.
3. Eğer eklenmediyse ekleriz, eklendiyse pas geçeriz.
4. İşlem bitince boşluk kalmaması adına, tam da benzersiz eleman sayısına eşit boyutta nihai bir dizi oluşturup döndürürüz.

```java
package denemee;
import java.util.Scanner;

public class EliminateDuplicates {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int[] numbers = new int[10];

        System.out.print("Enter ten numbers: ");
        for (int i = 0; i < numbers.length; i++) {
            numbers[i] = input.nextInt();
        }

        // Metodu cagirip benzersiz elemanlardan olusan yeni diziyi aliyoruz
        int[] distinctNumbers = eliminateDuplicates(numbers);

        System.out.print("The distinct numbers are: ");
        for (int num : distinctNumbers) {
            System.out.print(num + " ");
        }
    }
```

```java
    // Tekrarlari eleyip yeni bir dizi donduren metot
    public static int[] eliminateDuplicates(int[] list) {
        // En kotu senaryoda tum sayilar farkli olabilir, o yuzden ayni boyutta gecici dizi aciyoruz
        int[] temp = new int[list.length];
        int size = 0; // Gecici diziye kac tane benzersiz eleman eklendigini tutan sayac

        for (int i = 0; i < list.length; i++) {
            boolean isDuplicate = false; // Baslangicta elemani "kopya degil" kabul ediyoruz (Flag mantigi)

            // Eleman daha once temp dizisine eklenmis mi diye kontrol ediyoruz
            for (int j = 0; j < size; j++) {
                if (list[i] == temp[j]) {
                    isDuplicate = true; // Aynisini bulursak kopya oldugunu isaretle
                    break; // Donguyu kir, cunku bir tane bulmamiz yeterli
                }
            }

            // Eger kopya degilse temp dizisine ekle ve sayaci artir
            if (!isDuplicate) {
                temp[size] = list[i];
                size++;
            }
        }

        // Temp dizisinde bosluklar (sifirlar) kalmis olabilir.
        // Sadece dolu kismi (size kadar) iceren tam boyutlu yeni bir dizi olusturuyoruz.
        int[] result = new int[size];

        // System.arraycopy: (kaynak_dizi, baslangic_indeksi, hedef_dizi, hedef_baslangic, kopyalanacak_adet)
        System.arraycopy(temp, 0, result, 0, size);

        return result; // Temizlenmis diziyi dondur
    }
}
```

Row Sorting / Satır Sıralama

Kullanıcıdan bir matrisin satır ve sütun boyutlarını (örneğin 3x3, 4x5 gibi) girmesini isteyin. Belirtilen boyutlarda iki boyutlu bir dizi (matris) oluşturun ve içerisini Math.random() metodunu kullanarak rastgele ondalıklı (double) sayılarla doldurun.

Ardından, orijinal matrisin yapısını bozmadan, her bir satırının kendi içinde küçükten büyüğe sıralandığı yepyeni bir matris döndüren aşağıdaki metoda sahip bir program yazın:

```
public static double[ ][ ] sortRows(double[ ][ ] m)
```

(İpucu: Metot içerisinde satırları sıralamak için derste işlenen Arrays.sort( ) metodunu kullanabilirsiniz.)

Programınız çalıştığında önce üretilen rastgele matrisi, ardından da satırları sıralanmış yeni matrisi düzgün bir formatta ekrana yazdırmalıdır.

```java
package denemee;

import java.util.Arrays;
import java.util.Scanner;

public class RowSorting {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Matrisin satir sayisini girin: ");
        int rows = input.nextInt();
        System.out.print("Matrisin sutun sayisini girin: ");
        int cols = input.nextInt();

        double[][] matrix = new double[rows][cols];

        System.out.println("\nOlusturulan Rastgele Matris:");
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                // Math.random() 0.0 ile 1.0 arasi uretir, 10 ile carparak 0.0 - 10.0 araligina genisletiyoruz
                matrix[i][j] = Math.random() * 10;
                // %5.2f: Toplam 5 karakterlik yer ayir, virgulden sonra 2 basamak goster
                System.out.printf("%5.2f ", matrix[i][j]);
            }
            System.out.println();
        }

        // Siralama metodunu cagiriyoruz
        double[][] sortedMatrix = sortRows(matrix);

        System.out.println("\nSatirlari Siralanmis Yeni Matris:");
        for (int i = 0; i < sortedMatrix.length; i++) {
            for (int j = 0; j < sortedMatrix[i].length; j++) {
                System.out.printf("%5.2f ", sortedMatrix[i][j]);
            }
            System.out.println();
        }
    }
```

```java
    // Matrisin her bir satirini kendi icinde siralayan metot
    public static double[][] sortRows(double[][] m) {
        // Diziler referans tiplidir. Orijinal matrisi bozmamak icin ayni boyutlarda yepyeni bir matris olusturuyoruz
        double[][] result = new double[m.length][m[0].length];

        for (int i = 0; i < m.length; i++) {
            // Orijinal matrisin satirlarini sirayla yeni matrise kopyaliyoruz
            System.arraycopy(m[i], 0, result[i], 0, m[i].length);

            // Arrays.sort: Tek boyutlu dizileri siralar.
            // Matrislerde result[i] ifadesi "i. satiri" yani tek boyutlu bir diziyi temsil eder.
            Arrays.sort(result[i]);
        }
        return result;
    }
}
```

Çokgen Alt Alanları

Dışbükey (convex) bir dörtgenin 4 köşesinin birleştirilmesiyle ortada birbirini kesen iki köşegen oluşur. Bu köşegenler, dörtgeni 4 adet küçük üçgene böler.

Kullanıcıdan bu dörtgenin 4 köşesine ait (x, y) koordinatlarını (toplam 8 adet değer) girmesini isteyen bir program yazınız. Programınız, köşegenlerin kesişim noktasını bularak içeride oluşan bu 4 üçgenin alanını hesaplamalı ve sonuçları küçükten büyüğe doğru sıralayarak ekrana yazdırmalıdır.

Önemli Not (Püf Noktası): Kullanıcının girdiği 4 noktadan herhangi 3'ü (veya 4'ü birden) aynı doğru üzerinde (doğrusal / collinear) bulunuyorsa, ortada geçerli bir dörtgen oluşmayacağı için alan hesabı yapılamaz. Programınızın alan hesabına geçmeden önce bu durumu kontrol etmesi ve eğer noktalar doğrusal ise *"Alan hesabı yapılamaz, noktalar aynı çizgi üzerinde!"* şeklinde bir hata mesajı vererek çalışmayı sonlandırması gerekmektedir. Şekli Java ile görselleştirmenize gerek yoktur, sadece matematiksel olarak alanları bulmanız yeterlidir.

```java
public class PolygonSubareas {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter x1, y1, x2, y2, x3, y3, x4, y4: ");
        double[][] points = new double[4][2];
        for (int i = 0; i < 4; i++) {
            points[i][0] = input.nextDouble();
            points[i][1] = input.nextDouble();
        }

        // Noktaların 3'ü veya 4'ü aynı doğru üzerinde mi (collinear)?
        // Eger oyleyse dortgen olusmaz, alan hesabi yapilamaz.
        if (isCollinear(points[0], points[1], points[2]) ||
            isCollinear(points[0], points[1], points[3]) ||
            isCollinear(points[0], points[2], points[3]) ||
            isCollinear(points[1], points[2], points[3])) {
            System.out.println("Hata: Girilen noktalardan en az 3'ü aynı doğru üzerinde. Alan hesabı yapılamaz!");
            return; // Programı sonlandır
        }

        // Çapraz köşegenlerin kesişim (merkez) noktasını buluyoruz (Cramer Kurali ile)
        double[] intersect = getIntersectPoint(points[0], points[2], points[1], points[3]);

        if (intersect == null) {
            System.out.println("Köşegenler kesişmiyor, geçerli bir dışbükey dörtgen değil.");
            return;
        }
```

```java
        // Merkez noktayı kullanarak 4 adet üçgenin alanını hesapliyoruz
        double[] areas = new double[4];
        areas[0] = getTriangleArea(points[0], points[1], intersect);
        areas[1] = getTriangleArea(points[1], points[2], intersect);
        areas[2] = getTriangleArea(points[2], points[3], intersect);
        areas[3] = getTriangleArea(points[3], points[0], intersect);

        // Alanları küçükten büyüğe sırala
        Arrays.sort(areas);

        System.out.print("The areas are ");
        for (double area : areas) {
            System.out.printf("%.2f ", area);
        }
    }

```

```java
    // Üç noktanın aynı doğru üzerinde olup olmadığını alan hesaplayarak bulan metot
    public static boolean isCollinear(double[] p1, double[] p2, double[] p3) {
        double area = getTriangleArea(p1, p2, p3);
        // double kusuratindan dolayi 0 yerine cok kucuk bir sayi cikabilir.
        // 1e-9 (0.000000001) degerinden kucukse sıfır (dogrusal) kabul ediyoruz.
        return area < 1e-9;
    }

    // Analitik geometri ile 3 noktasi bilinen ucgenin alanini hesaplayan metot
    public static double getTriangleArea(double[] p1, double[] p2, double[] p3) {
        // Math.abs (Mutlak Deger): Alan negatif olamayacagi icin sonucu hep pozitife cevirir
        return Math.abs((p1[0] * (p2[1] - p3[1]) + p2[0] * (p3[1] - p1[1]) + p3[0] * (p1[1] - p2[1])) / 2.0);
    }
```

```java
    // Cramer kuralı (Denklem Sistemleri) ile 2 doğrunun kesişim noktasını bulan metot
    public static double[] getIntersectPoint(double[] p1, double[] p2, double[] p3, double[] p4) {
        double a = p1[1] - p2[1];
        double b = p2[0] - p1[0];
        double e = (p1[1] - p2[1]) * p1[0] + (p2[0] - p1[0]) * p1[1];

        double c = p3[1] - p4[1];
        double d = p4[0] - p3[0];
        double f = (p3[1] - p4[1]) * p3[0] + (p4[0] - p3[0]) * p3[1];

        double determinant = a * d - b * c;

        // Determinant 0 ise dogrular paraleldir, hicbir zaman kesismezler.
        if (Math.abs(determinant) < 1e-9) {
            return null;
        }

        // x ve y kesisim koordinatlarini hesapla
        double x = (e * d - b * f) / determinant;
        double y = (a * f - e * c) / determinant;

        return new double[]{x, y}; // Yeni bir dizi icinde x ve y'yi dondur
    }
}
```